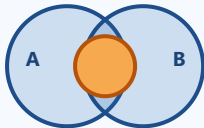


01 ¿Qué es un JOIN?

Es una operación que **combina filas de dos o más tablas** a partir de una condición de relación, normalmente entre una **llave foránea (FK)** y una **llave primaria (PK)**. En comercial_db: clientes.id_ciudad → ciudades.id_ciudad, productos.id_categoria → categorias.id_categoria, empleados.id_jefe → empleados.id_empleado.

02 Los seis tipos de JOIN

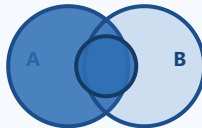
INNER JOIN



¿Cuándo? Solo las filas que **coinciden** en ambas tablas. El más usado: clientes que sí tienen ciudad, productos con su categoría.

```
FROM clientes AS c
INNER JOIN ciudades AS ci ON c.id_ciudad =
ci.id_ciudad
```

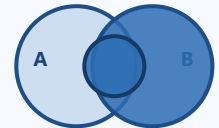
LEFT JOIN



¿Cuándo? **Todas** las filas de A + coincidencias; si no hay, **NULL** a la derecha. Para ver registros "huérfanos".

```
FROM productos AS p
LEFT JOIN categorias AS ca ON
p.id_categoria = ca.id_categoria
```

RIGHT JOIN



¿Cuándo? **Todas** las filas de B + coincidencias; si no hay, **NULL** a la izquierda. Es el espejo del LEFT JOIN.

```
FROM clientes AS c
RIGHT JOIN ciudades AS ci ON c.id_ciudad =
ci.id_ciudad
```

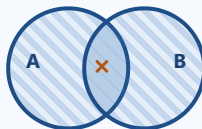
FULL OUTER JOIN



¿Cuándo? **Todas** las filas de ambas tablas con sus coincidencias; lo que no coincide queda con NULL. **SQL Server SÍ lo soporta** (MySQL no).

```
FROM empleados AS e
FULL OUTER JOIN departamentos AS d ON
e.id_departamento = d.id_departamento
```

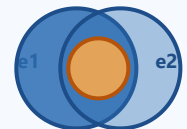
CROSS JOIN



¿Cuándo? **Producto cartesiano**: todas las filas de A × todas las de B. **No lleva ON**. Se usa poco: combinaciones posibles.

```
FROM productos AS p
CROSS JOIN categorias AS ca
```

SELF JOIN



¿Cuándo? La tabla se une **consigo misma** con dos alias. Ej.:

```
empleados.id_jefe
apunta a otro
empleados.id_empleado
: jefe y subordinado.

FROM empleados AS e
LEFT JOIN empleados AS j ON e.id_jefe =
j.id_empleado
```

03 Ejemplo aplicado — comercial_db

Nota: el esquema usa id_categoria NOT NULL, por lo que todos los productos tienen categoría; aún así, el LEFT JOIN es la herramienta correcta para detectar registros sin correspondencia.

```
-- INNER: clientes con su ciudad y su estado
SELECT c.nombre AS cliente, ci.nombre AS ciudad, e.nombre AS estado
FROM clientes AS c
INNER JOIN ciudades AS ci ON c.id_ciudad = ci.id_ciudad
INNER JOIN estados AS e ON ci.id_estado = e.id_estado;

-- SELF JOIN: empleados con el nombre de su jefe (LEFT para incluir jefes sin jefe)
SELECT e.nombre AS empleado, j.nombre AS jefe
FROM empleados AS e
LEFT JOIN empleados AS j ON e.id_jefe = j.id_empleado;
```

Conclusión: los JOIN son la base de las **consultas relacionales multi-tabla**: sin ellos no es posible reconstruir información completa que el modelo relacional guarda separada, como un cliente con su ciudad o un producto con su categoría.

Fuentes:

- Microsoft Learn — SQL Server: JOIN (Transact-SQL) y combinaciones de tablas.
- W3Schools — SQL Tutorial: SQL JOIN, LEFT, RIGHT, FULL OUTER y CROSS JOIN.
- Silberschatz, Korth y Sudarshan — Fundamentos de Bases de Datos (álgebra relacional: la reunión o join).